# SMART CAMPUS & PLACEMENT PLATFORM

Project Documentation & Technical Blueprint

A progressive full-stack application designed to demonstrate practical Java backend engineering from fundamentals to production readiness.

| | |
|---|---|
| **Project Owner** | Aryan Nagori |
| **Current Team** | 1 member — Aryan Nagori |
| **Primary Focus** | Java Backend Development |
| **Current Stage** | Project setup / architecture planning |
| **Backend** | Spring Boot + Spring MVC + REST |
| **Persistence** | JPA + Hibernate + Spring Data JPA |
| **Database** | PostgreSQL |
| **Build Tool** | Maven |
| **Planned Frontend** | React |

Version 1.1 • September 2026

# 1. Executive Overview

The Smart Campus & Placement Platform is a full-stack application whose primary purpose is to provide a realistic, evolving backend engineering project. It starts with a simple Spring Boot REST API and progressively grows into a production-style system with layered architecture, relational data modeling, validation, exception handling, authentication, authorization, testing, containerization, deployment, and a React frontend.

> The project is intentionally designed as a learning vehicle. Each technology is introduced because it solves a concrete engineering problem, not because an annotation or framework feature needs to be memorized.

## Core objective

- Build a realistic backend rather than a basic CRUD demo.
- Connect Spring Boot, REST, JPA/Hibernate, PostgreSQL, security, testing, Docker, and deployment into one coherent system.
- Understand why each architectural layer, annotation, library, and design decision exists.
- Create a portfolio project that can be explained clearly in interviews from HTTP request to database and back.

# 2. What the Product Does

The platform connects students, companies, jobs, applications, skills, projects, assessments, and placement readiness. It is designed around a real campus-placement workflow rather than only storing student records.

## Student side

- Create and manage a student profile.
- Maintain skills and projects.
- Track placement readiness and assessment results.
- Browse relevant jobs and submit applications.
- Track application status and recruitment progress.

## Company / recruiter side

- Create a company profile.
- Create job postings and define requirements.
- Review applications.
- Shortlist candidates and manage interview stages.
- Record selection or rejection outcomes.

## Platform intelligence

- Calculate or represent a student placement-readiness score.
- Match student skills against job requirements.
- Recommend jobs based on skills and readiness.
- Provide useful summaries and dashboards in the later frontend phase.

# 3. Why This Project Is Being Built

The project is intentionally cumulative. Earlier learning—Maven, Servlets/Tomcat, Spring Core, REST, Hibernate, JPA, and PostgreSQL—forms the foundation. The same application will then be upgraded instead of starting disconnected mini-projects for every technology.

This approach demonstrates progression: basic implementation → clean architecture → optimization and production concerns. It also makes Git history meaningful because each commit represents a real engineering milestone.

# 4. Target Architecture

The final backend will follow a layered architecture with clear separation of responsibilities.

```
Client / React Frontend
↓ HTTP + JSON
Spring Security / Authentication
↓
Controller Layer → DTO / Request & Response Models
↓
Service Layer → Business Logic
↓
Repository Layer → Data Access
↓
Spring Data JPA → JPA → Hibernate
↓
PostgreSQL
```

## Responsibilities

- **Controller:** receives HTTP requests, handles the API boundary, calls application services, and produces HTTP responses.
- **DTO:** defines the API contract without exposing persistence entities directly.
- **Service:** contains business rules and coordinates operations across repositories.
- **Repository:** provides persistence operations and database queries.
- **Entity:** represents persistent domain data and its relationships.
- **JPA/Hibernate:** maps Java objects to relational database structures and manages persistence.
- **PostgreSQL:** stores durable application data.
- **Spring Security:** protects endpoints and manages authentication/authorization in later phases.

## Request flow

### Example: POST /students

React/Postman → HTTP request → Spring MVC/DispatcherServlet → StudentController → StudentService → StudentRepository → Spring Data JPA → Hibernate → JDBC → PostgreSQL → response back to client.

# 5. Planned Domain Model

The initial domain will grow gradually. The entities below represent the planned major business concepts.

| Entity | Purpose | Key relationships |
|---|---|---|
| Student | Student profile and placement information | Skills, Projects, Applications |
| Skill | Technology/skill associated with a student | Student; later Job requirements |
| Project | Projects completed by a student | Student |
| Company | Organization recruiting candidates | Jobs, Recruiters |
| Job | Open position and its requirements | Company, Applications |
| Application | Student's application to a job | Student, Job |
| Assessment | Skill/readiness evaluation | Student; skills/results |
| Interview | Recruitment stage and interview information | Application |
| User / Account | Authentication identity | Roles and domain profile |

# 6. Technology Stack

| Layer | Technology | Role |
|---|---|---|
| Language | Java | Primary backend language |
| Build | Maven | Dependencies and build lifecycle |
| Framework | Spring Boot | Application foundation and configuration |
| Web | Spring MVC / Spring Web | REST API and HTTP request handling |
| Persistence | JPA + Hibernate | ORM and persistence |
| Data access | Spring Data JPA | Repository abstraction and queries |
| Database | PostgreSQL | Relational data storage |
| Security | Spring Security + JWT | Authentication and authorization |
| Testing | JUnit + Mockito + Spring testing | Automated testing |
| Containerization | Docker + Docker Compose | Repeatable local/production environments |
| Frontend | React | User-facing web application |
| API testing | Postman / equivalent | Manual API testing during backend development |
| Version control | Git + GitHub | Source control and portfolio history |

# 7. Development & Learning Method

The project will be developed one meaningful file or small feature at a time. The objective is understanding, not code copying.

**Concept → Why it exists → Simple example → Small implementation → Important points → Project usage → Test → Review → Move forward**

For annotations, the focus will be on the problem they solve. For example, @Entity will be understood in terms of persistence mapping; @Autowired in terms of dependency injection; @RestController in terms of Spring MVC request handling. The same approach applies to JPA relationships, transactions, security filters, validation, testing, and Docker.

# 8. Development Roadmap

| Phase | Milestone | Main learning outcomes |
|---|---|---|
| 0 | Project setup + Git | Spring Initializr, Maven, project structure, Git workflow |
| 1 | Spring Boot foundation | SpringApplication, @SpringBootApplication, auto-configuration, component scanning, configuration |
| 2 | Student CRUD | Controller → Service → Repository → JPA → PostgreSQL |
| 3 | Professional architecture | Packages, DTOs, mappers, dependency boundaries |
| 4 | Validation + errors | Bean Validation, global exception handling, consistent API errors |
| 5 | Relationships | One-to-one, one-to-many, many-to-one, many-to-many, cascade, lazy/eager, N+1 |
| 6 | Queries | JPQL, custom queries, filtering, pagination, sorting, indexes |
| 7 | Transactions | @Transactional, atomicity, rollback, service-level transaction boundaries |
| 8 | Security | Spring Security, password hashing, authentication, authorization, JWT, roles |
| 9 | Testing | JUnit, Mockito, MockMvc, integration tests |
| 10 | Production concerns | Logging, profiles, configuration, API documentation, health checks |
| 11 | Docker | Dockerfile, images, containers, Compose, environment variables |
| 12 | Deployment | Deploy backend and database using a practical cloud setup |
| 13 | React frontend | Student/company/admin UI consuming the REST API |
| 14 | Final polish | README, architecture diagram, screenshots, API examples, resume-ready documentation |

# 9. Git & GitHub Strategy

Git will be used throughout development rather than only after the project is complete. Commits should represent meaningful, explainable milestones.

## Example commit progression

```
Initial Spring Boot project
Add Student entity
Add Student repository
Add Student service
Add Student REST endpoints
Add validation for student requests
Add global exception handling
Add student-project relationship
Add job and application domain
Add authentication and JWT authorization
Add automated tests
Dockerize application
```

The main branch should remain a stable version. Feature branches can be used for meaningful features, followed by a merge into main. Secrets such as database passwords and JWT secrets must never be committed.

# 10. Frontend Plan

The frontend will be added after the backend API is stable enough to consume. React is the planned frontend technology. The frontend is intentionally secondary to the Java backend learning objective.

## Planned frontend areas

- Student dashboard: profile, skills, projects, readiness, recommended jobs, applications.
- Company/recruiter dashboard: company profile, jobs, applicants, interview pipeline.
- Admin dashboard: platform management and useful operational summaries.
- Authentication screens: registration, login, protected routes, role-aware navigation.
- API integration: JSON-based communication with the Spring Boot backend.

# 11. API Direction

The initial API will start with conventional CRUD endpoints and grow as the domain grows.

```
GET /students
POST /students
GET /students/{id}
PUT /students/{id}
DELETE /students/{id}
```

Later endpoint groups will include /auth, /skills, /projects, /companies, /jobs, /applications, /assessments, /interviews, and user/profile operations. Exact API contracts will be designed as the corresponding feature is implemented.

# 12. Engineering Principles

- Separation of concerns: each layer has a clear responsibility.
- Dependency Injection over manual dependency construction.
- Entities are not automatically treated as API contracts; DTOs will be introduced when appropriate.
- Business logic belongs in services rather than controllers.
- Database operations should have deliberate transaction boundaries.
- Errors should be predictable and useful to API consumers.
- Security must be designed into the application rather than added as an afterthought.
- Tests should protect important business behavior.
- Configuration and secrets should be externalized for deployment.
- Performance improvements should be based on understanding the actual problem, not premature optimization.

# 13. Current Team

**Project owner / developer:** Aryan Nagori

The project currently has one member. The architecture and documentation are being designed so that it can still resemble a professional team project, with clear modules, responsibilities, Git history, documentation, and future extensibility.

# 14. Current Status

- Spring Boot project created using Maven.
- Spring Web/MVC dependency added.
- Spring Data JPA dependency added.
- PostgreSQL driver added.
- Spring Core fundamentals already studied: IoC, DI, beans, @Component, @Autowired, @Qualifier, @Primary, XML configuration.
- REST CRUD fundamentals already practiced.
- Direct Hibernate and Spring Data JPA have both been practiced.
- PostgreSQL has already been used with Hibernate/JPA.
- Next immediate step: understand the generated Spring Boot project and main application class before implementing the first domain feature.

# 15. Definition of Done

> At completion, the project should be more than a working application. It should be an interview-explainable system in which the developer can trace a request through Spring MVC, controllers, DTOs, services, repositories, JPA/Hibernate, JDBC, PostgreSQL, security, transactions, testing, and deployment.

The final deliverable should include a working backend, a React frontend, a documented API, a clean GitHub repository, database design, tests, Docker configuration, deployment instructions, and a README that explains architectural decisions.

# 16. Immediate Next Step

Do not create all project classes at once. Start with the generated Spring Boot application. Understand the project structure and the main application class, especially what SpringApplication.run() does. Then introduce the first domain feature one file at a time.